

Implementación del Proceso Unificado con IA

Pomora y trazabilidad de dos incrementos funcionales

Tarea 3 · Grupo 06 · GR06_1BT3_622_26B

Este informe presenta la implementación Java Web del temporizador individual y del estudio compartido. Relaciona los modelos con las clases, los mensajes de secuencia, la persistencia y las evidencias de funcionamiento.

Integrantes

Doménica Anabell Astudillo Ayala

Ariel Geovanny Baldeón Cervantes

Mayerli Anahi Chavez Toapanta

Camila Monserrath Paredes Rodríguez

Profesor: Prof. Julio Sandovalín Ph.D

Fecha de entrega: 4 de octubre de 2026

Repositorio: https://github.com/MayerliChavez/GR06_1BT3_622_26B

Índice

Sección	Página
1 Objetivos y fundamento del proceso	3
2 Arquitectura y entorno de ejecución	4
3 Modelo de clases del proyecto	5
4 Correspondencia de clases y relaciones	6
5 Matriz de trazabilidad de los casos de uso	7
6 Incremento 1 CU01 Iniciar temporizador	8
7 Incremento 1 CU02 Pausar temporizador	9
8 Incremento 1 CU03 Reanudar temporizador	10
9 Incremento 1 CU04 Consultar historial	11
10 Incremento 2 CU01 Registrar usuario	12
11 Código y persistencia del registro	13
12 Incremento 2 CU02 Iniciar sesión	14
13 Incremento 2 CU03 Emparejar compañero	15
14 Código y continuidad del estudio compartido	16
15 Evidencias del temporizador individual	17
16 Evidencias del historial y las cuentas	18
17 Evidencias del ingreso y emparejamiento	19
18 Verificación y resultados reproducibles	20
19 Publicación y herramientas de desarrollo	21
20 Conclusiones y bibliografía	22

1 Objetivos y fundamento del proceso

Objetivo general

Implementar y documentar los dos incrementos funcionales de Pomora mediante Java Web con JSP, Servlet y ORM, manteniendo la correspondencia entre los modelos y el comportamiento del sistema.

Objetivos específicos

- Relacionar los casos de uso con las clases, métodos y asociaciones que los realizan.
- Explicar las interacciones del flujo principal y su correspondencia con el código Java.
- Verificar las reglas temporales, la persistencia, el registro y el emparejamiento mediante pruebas y evidencias de pantalla.

Marco teórico

El Proceso Unificado organiza el desarrollo de manera iterativa e incremental, dirigido por casos de uso y centrado en la arquitectura. Un incremento agrega comportamiento observable y conserva lo construido en los anteriores. En Pomora, el primer incremento permite estudiar individualmente; el segundo incorpora cuentas y coordinación con otro estudiante.

El diagrama de clases define la estructura estática: responsabilidades, atributos, operaciones y relaciones. El diagrama de secuencia muestra el orden de las colaboraciones para realizar un caso de uso. La trazabilidad vincula cada elemento modelado con su implementación y su verificación; permite localizar el código que realiza un mensaje concreto y comprobar su efecto.

JSP genera la interfaz HTML; Servlet recibe solicitudes HTTP y prepara respuestas; los servicios coordinan los casos de uso. Jakarta Persistence define el mapeo objeto relacional y Hibernate lo implementa. Las entidades del dominio se almacenan en H2, mientras que las sesiones HTTP identifican al estudiante autenticado.

Organización de la lectura

Las secciones siguientes describen la arquitectura, la correspondencia de clases y los siete casos de uso. Cada caso incluye una secuencia del flujo principal y referencias verificables al código. Después se presentan evidencias, pruebas, publicación y conclusiones.

2 Arquitectura y entorno de ejecución

Capa	Componentes	Responsabilidad
Presentación	acceso.jsp, pomora.jsp, compartido.jsp	Formularios, temporizador, historial y compañero.
Control HTTP	AccesoServlet, PomodoroServlet, CompartidoServlet	Adaptar solicitudes, validar CSRF y preparar vistas.
Aplicación	ServicioCuentas, ServicioPomodoro, ServicioEstudioCompartido	Coordinar registro, autenticación y estudio.
Dominio	Estudiante, CuentaUsuario, sesiones y bloques	Conservar identidad, estados y reglas temporales.
Persistencia	Repositorios JPA y EntityManager	Consultar y guardar en transacciones.
Infraestructura	PersistenceListener, AccesoFilter, PresenciaCompartida	Inicialización, acceso autenticado y presencia.

Tecnologías configuradas

pom.xml configura Java 17 como nivel de compilación, empaquetado WAR, Jakarta Servlet 6.0.0, Hibernate ORM 6.6.58.Final, H2 2.3.232, JSTL y JUnit Jupiter 5.11.4. Jetty 12.0.35 ejecuta el proyecto en desarrollo. No se confunde el servidor utilizado para las comprobaciones con el IDE exigido por la tarea.

Identidad y almacenamiento

La identidad procede de la sesión HTTP autenticada: cuentald y estudianteld. Cada cuenta está asociada a un Estudiante. persistence.xml usa jdbc:h2:file:./data/pomora; las pruebas usan bases H2 en memoria independientes. La cola de emparejamiento permanece en memoria y no constituye una entidad adicional.

Correspondencia del modelo actualizado

El diagrama de clases actualizado suministrado por el grupo incluye CuentaUsuario, ServicioCuentas y EntityManagerFactory. Por tanto, registro y autenticación forman parte de la referencia actual. Los getters, constructores sin argumentos para JPA y adaptadores web son infraestructura adicional; la correspondencia funcional no significa igualdad literal de todas las declaraciones.

3 Modelo de clases del proyecto

Incremento 2: Emparejamiento y estudio compartido

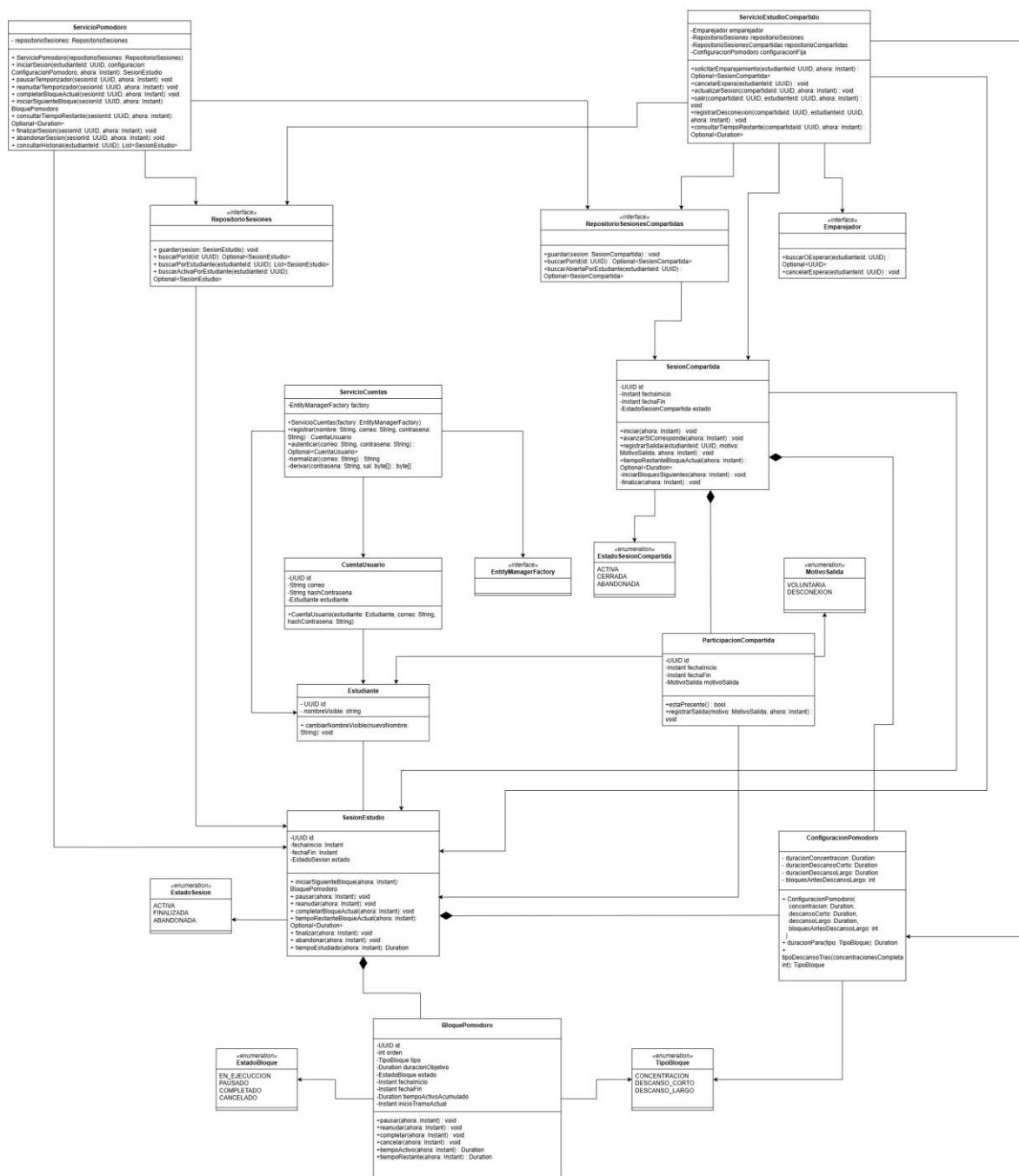


Figura 1. Modelo de clases actualizado aportado por el grupo. Incluye los dos incrementos y las cuentas.

En Word se puede ampliar la imagen para consultar firmas. Las tablas siguientes detallan las correspondencias relevantes sin depender de la legibilidad del modelo completo.

4 Correspondencia de clases y relaciones

Elemento UML	Implementación y relación
Estudiante	UUID id, nombreVisible y cambiarNombreVisible(String). Entidad estudiante.
CuentaUsuario	UUID id, correo, hashContrasena y estudiante. @OneToOne con cascada PERSIST; correo único.
ServicioCuentas	EntityManagerFactory factory; registrar, autenticar; normalizar y derivar privados.
SesionEstudio	Estudiante mediante @ManyToOne; configuración @Embedded; bloques @OneToMany con cascada y orden.
BloquePomodoro	Orden, tipo, duración, estado e instantes. Pausar, reanudar, completar, cancelar y calcular tiempos.
ConfiguracionPomodoro	@Embeddable; tres duraciones y frecuencia del descanso largo. duracionPara y tipoDescansoTras.
SesionCompartida	Composición de sesiones individuales y participaciones mediante @OneToMany. iniciarBloquesSiguients y finalizar son privados.
ParticipacionCompartida	Asocia Estudiante y SesionEstudio; conserva fecha de salida y MotivoSalida.
RepositorioSesiones	guardar, buscarPorId, buscarPorEstudiante y buscarActivaPorEstudiante; implementación Jpa.
RepositorioSesionesCompartidas	guardar, buscarPorId y buscarAbiertaPorEstudiante; implementación Jpa.
Emparejador	buscarOEsperar y cancelarEspera; EmparejadorEnMemoria mantiene la cola y selecciona al azar.

Las enumeraciones coinciden con el modelo: EstadoSesion, EstadoBloque, TipoBloque, EstadoSesionCompartida y MotivoSalida. Una sesión individual sigue ACTIVA cuando su bloque está PAUSADO; no se introduce un estado PAUSADA en EstadoSesion.

5 Matriz de trazabilidad de los casos de uso

Caso	Entrada y vista	Operación principal	Resultado observable
I1 CU01 Iniciar	POST /pomodoro; pomora.jsp	ServicioPomodoro.iniciarSesion	Primer bloque en ejecución.
I1 CU02 Pausar	POST /pomodoro; accion=pausar	pausarTemporizador → pausar	Tiempo restante conservado.
I1 CU03 Reanudar	POST /pomodoro; accion=reanudar	reanudarTemporizador → reanudar	Nuevo tramo activo.
I1 CU04 Historial	GET /historial; pomora.jsp	consultarHistorial	Sesiones y bloques de la cuenta.
I2 CU01 Registro	POST /registro; acceso.jsp	ServicioCuentas.registrar	Cuenta guardada y confirmación.
I2 CU02 Ingreso	POST /ingresar; acceso.jsp	ServicioCuentas.autenticar	Sesión autenticada y /compartido.
I2 CU03 Emparejar	POST /compartido; accion=buscar	solicitarEmparejamiento	Dos participantes sincronizados.

Criterio de las secuencias

Los gráficos de este informe describen únicamente el flujo principal. Se completaron y contrastaron con el código revisado; no se atribuyen mensajes a los cuadros de secuencia vacíos de las referencias anteriores. Las validaciones permanecen en el código aunque no se dibujen ramas alternativas.

Las etiquetas de interfaz, consultas y confirmaciones describen interacciones. Los mensajes con nombres de métodos se corresponden con las operaciones Java. EntityManagerFactory crea EntityManager; no ejecuta normalizar, derivar ni persist. Las tablas de cada caso identifican los archivos y líneas que permiten comprobarlo.

6 Incremento 1 CU01 Iniciar temporizador

Precondición: estudiante autenticado, sin sesión activa y sin estudio compartido abierto. El controlador proporciona configuración 25/5/15 y descanso largo cada cuatro concentraciones. La sesión crea el primer BloquePomodoro de concentración.

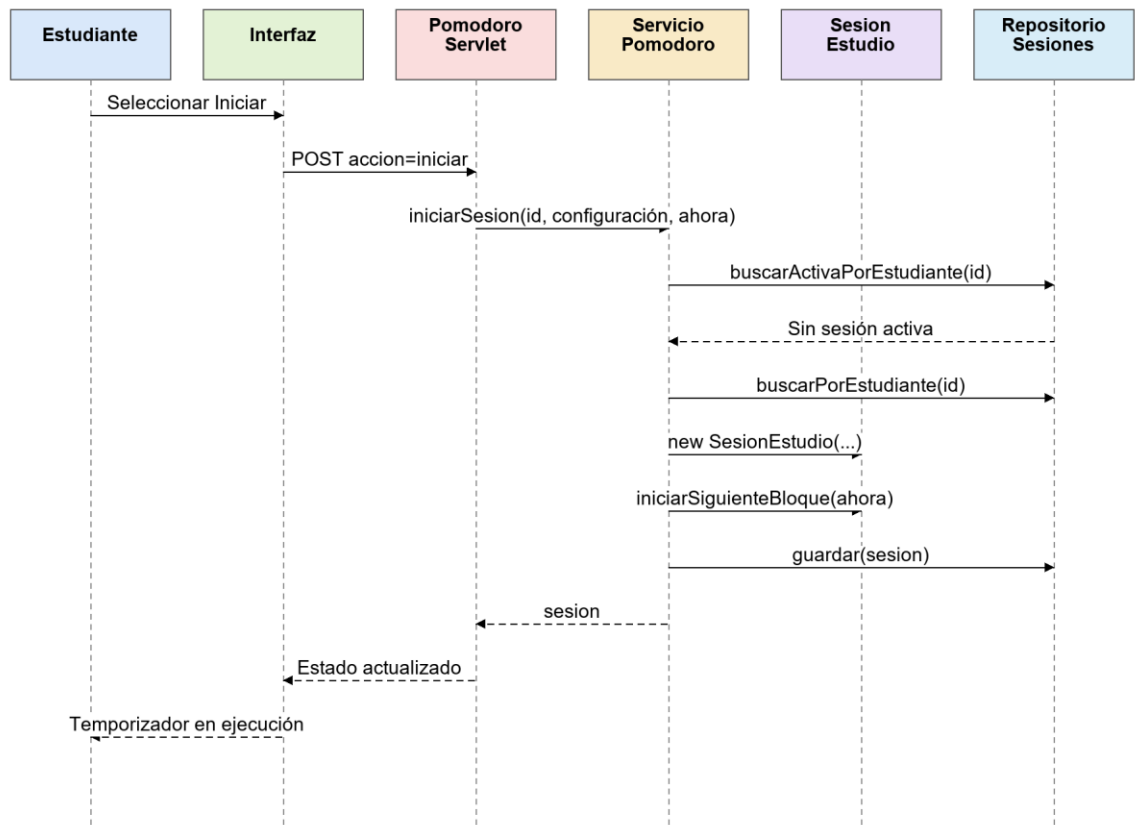


Figura 2. Secuencia del flujo principal. Fuente: modelo y código del proyecto.

Mensaje u operación	Ubicación en el código
iniciarSesion(UUID, ConfiguracionPomodoro, Instant)	ServicioPomodoro.java:15
iniciarSiguienteBloque(Instant)	SesionEstudio.java:32
guardar(SesionEstudio)	RepositorioSesionesJpa.java:12

Fragmento de coordinación — src/main/java/edu/proyecto/service/ServicioPomodoro.java, líneas 20–23.

```
var sesion = new SesionEstudio(estudiante, configuracion, ahora);
sesion.iniciarSiguienteBloque(ahora);
repositorioSesiones.guardar(sesion);
return sesion;
```

7 Incremento 1 CU02 Pausar temporizador

Precondición: bloque individual EN_EJECUCION con tiempo pendiente. BloquePomodoro acumula el tiempo activo y elimina el inicio del tramo. La sesión permanece ACTIVA. La flecha de interfaz resume el POST y la delegación del Servlet.

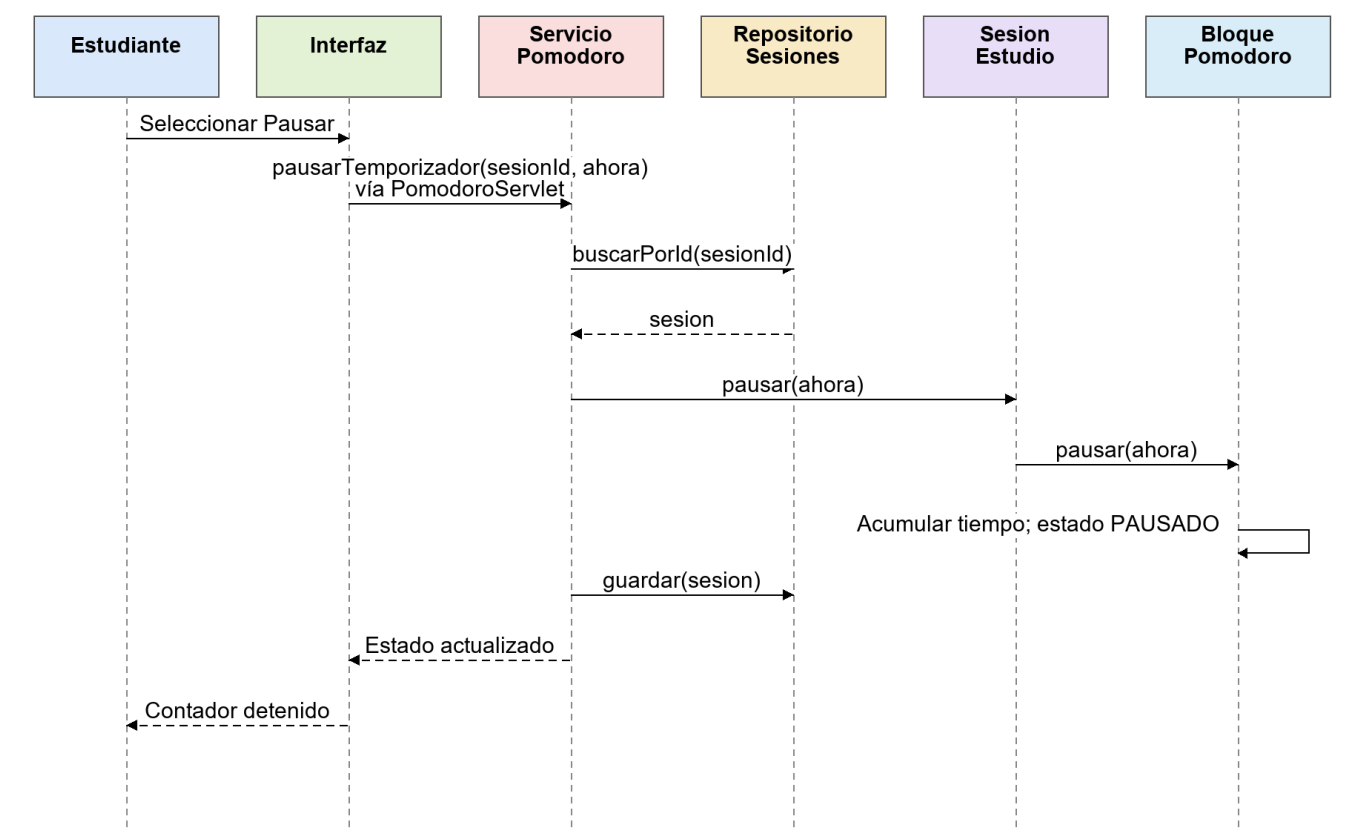


Figura 3. Secuencia del flujo principal. Fuente: modelo y código del proyecto.

Mensaje u operación	Ubicación en el código
pausarTemporizador(UUID, Instant)	ServicioPomodoro.java:26
SesionEstudio.pausar(Instant)	SesionEstudio.java:50
BloquePomodoro.pausar(Instant)	BloquePomodoro.java:37

Fragmento del cambio de estado — src/main/java/edu/proyecto/model/BloquePomodoro.java, líneas 42–44.

```
tiempoActivoAcumulado = tiempoActivo(ahora);
inicioTramoActual = null;
estado = EstadoBloque.PAUSADO;
```

8 Incremento 1 CU03 Reanudar temporizador

Precondición: bloque PAUSADO y sesión ACTIVA. Reanudar conserva tiempoActivoAcumulado y establece inicioTramoActual con el instante recibido. El intervalo pausado no se suma al tiempo de concentración.

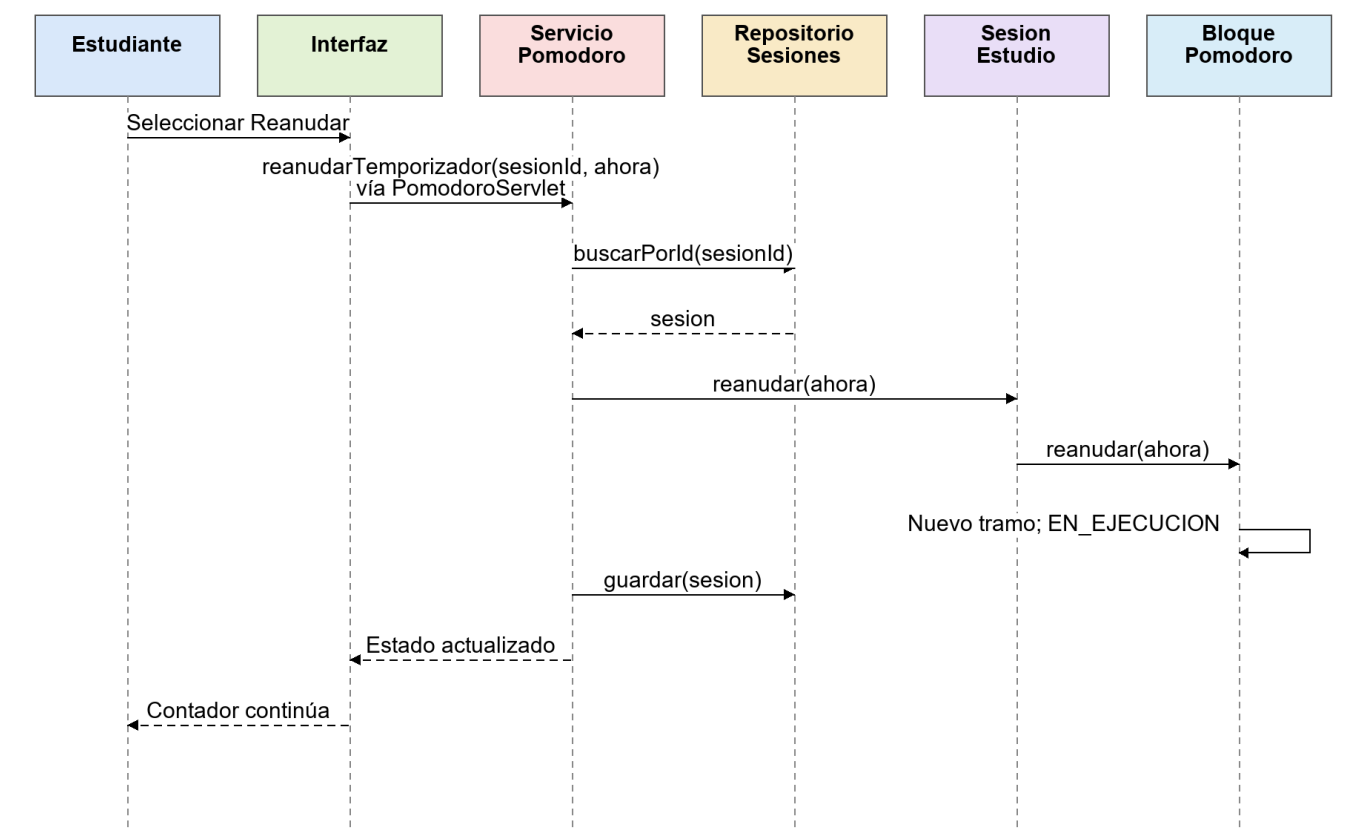


Figura 4. Secuencia del flujo principal. Fuente: modelo y código del proyecto.

Mensaje u operación	Ubicación en el código
reanudarTemporizador(UUID, Instant)	ServicioPomodoro.java:32
SesionEstudio.reanudar(Instant)	SesionEstudio.java:51
BloquePomodoro.reanudar(Instant)	BloquePomodoro.java:47

Fragmento del cambio de estado — src/main/java/edu/proyecto/model/BloquePomodoro.java, líneas 47–52.

```
public void reanudar(Instant ahora) {
    validarAhora(ahora);
    if (estado != EstadoBloque.PAUSADO)
        throw new IllegalStateException("Solo se puede reanudar un bloque pausado.");
    inicioTramoActual = ahora;
    estado = EstadoBloque.EN_EJECUCION;
```

9 Incremento 1 CU04 Consultar historial

Precondición: estudiante autenticado. prepararVista construye las filas del historial con fecha, estado, bloques y tiempo estudiado. La consulta se ordena por inicio descendente y filtra por la identidad de la cuenta.

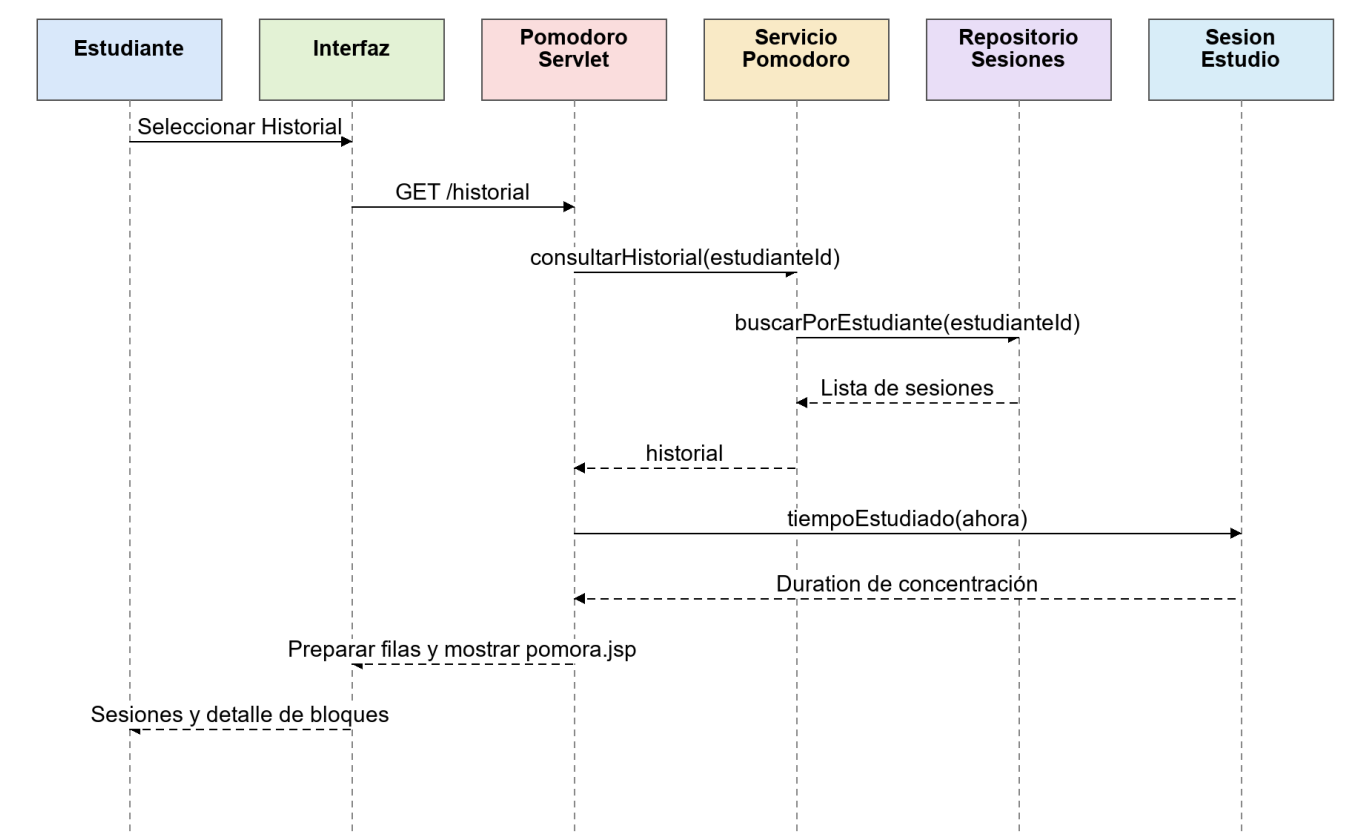


Figura 5. Secuencia del flujo principal. Fuente: modelo y código del proyecto.

Mensaje u operación	Ubicación en el código
consultarHistorial(UUID)	ServicioPomodoro.java:68
buscarPorEstudiante(UUID)	RepositorioSesionesJpa.java:33
tiempoEstudiado(Instant)	SesionEstudio.java:87

Fragmento del cálculo de concentración — src/main/java/edu/proyecto/model/SesionEstudio.java, líneas 87–89.

```
public Duration tiempoEstudiado(Instant ahora) {
    return bloques.stream().filter(b -> b.getTipo() == TipoBloque.CONCENTRACION)
        .map(b -> b.tiempoActivo(ahora)).reduce(Duration.ZERO, Duration::plus);
}
```

10 Incremento 2 CU01 Registrar usuario

Precondición: nombre y correo válidos, contraseña de 8 a 128 caracteres y correo disponible. El servicio obtiene EntityManager mediante factory.createEntityManager(). El registro guarda cuenta y estudiante, pero no autentica automáticamente.

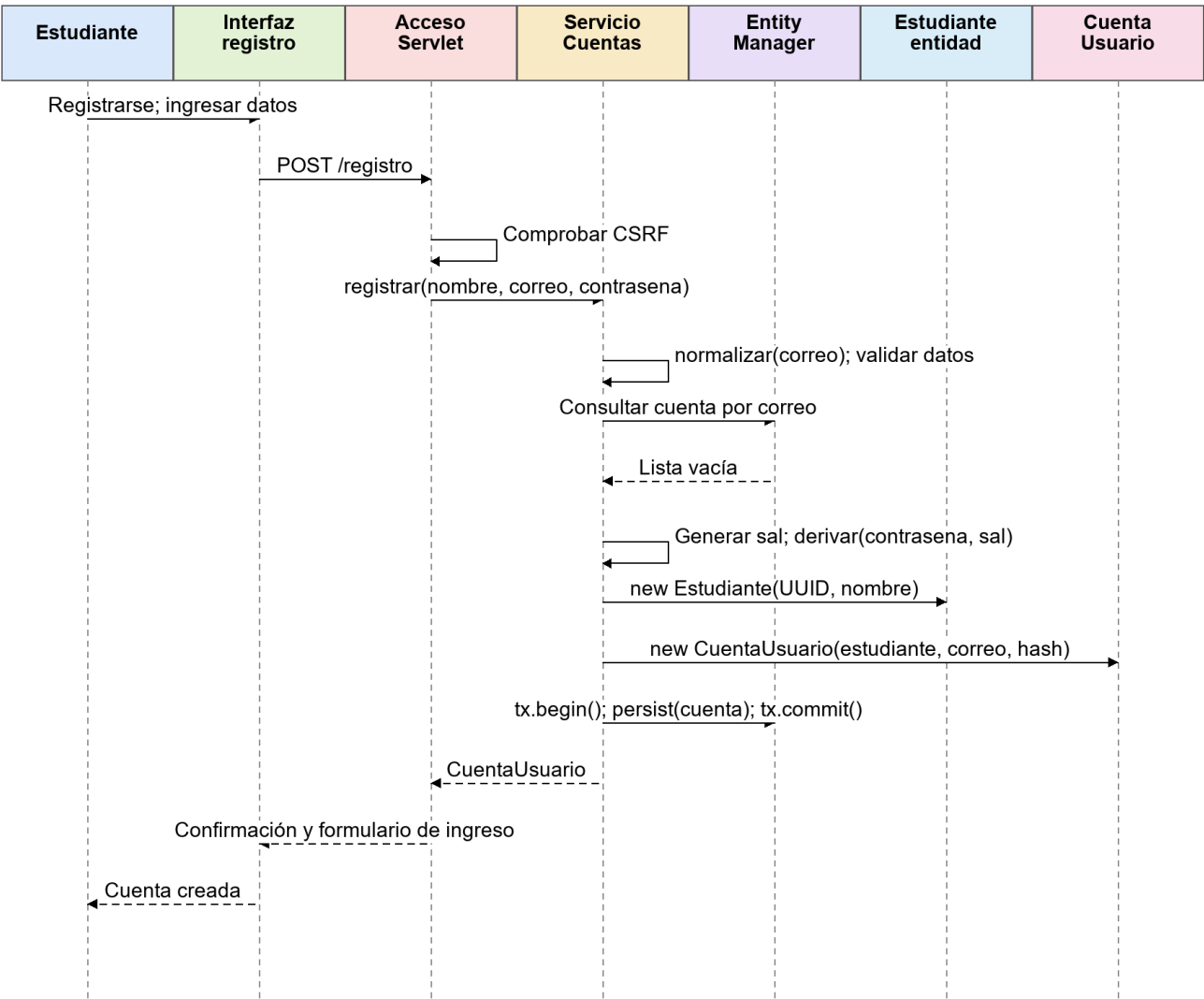


Figura 6. Secuencia del flujo principal. Fuente: modelo y código del proyecto.

Mensaje u operación	Ubicación en el código
registrar(String, String, String)	ServicioCuentas.java:13
normalizar(String) / derivar(String, byte[])	ServicioCuentas.java:39 / ServicioCuentas.java:40
Confirmación y registro=false	AccesoServlet.java:40

11 Código y persistencia del registro

La firma del constructor del modelo es CuentaUsuario(Estudiante, String, String). El identificador de la cuenta se obtiene del estudiante asociado. No se agrega un parámetro id inexistente en el constructor.

Fragmento exacto del registro — src/main/java/edu/proyecto/service/ServicioCuentas.java, líneas 21–26.

```
byte[] sal=new byte[16]; new SecureRandom().nextBytes(sal);
String
hash="600000:"+Base64.getEncoder().encodeToString(sal)+":"+Base64.getEncoder().encodeToString(derivar(contrasena,sal
));
var cuenta=new CuentaUsuario(new Estudiante(UUID.randomUUID(),nombre),correo,hash);
var tx=em.getTransaction();
try { tx.begin(); em.persist(cuenta); tx.commit(); return cuenta; }
catch (RuntimeException e) { if(tx.isActive())tx.rollback(); throw e; }
```

Correspondencia de la contraseña

derivar devuelve byte[], no String. El servicio codifica la sal y la clave derivada en Base64 y construye hashContrasena con el número de iteraciones. normalizar y derivar son métodos privados del servicio; sus mensajes son llamadas al mismo objeto, no a EntityManagerFactory.

Fragmento exacto de la derivación — src/main/java/edu/proyecto/service/ServicioCuentas.java, líneas 40–44.

```
private byte[] derivar(String contrasena, byte[] sal) {
    var spec=new PBEKeySpec(contrasena.toCharArray(),sal,600000,256);
    try { return SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(spec).getEncoded(); }
    catch (GeneralSecurityException e) { throw new IllegalStateException("No se pudo procesar la contraseña.",e);
}
    finally { spec.clearPassword(); }
```

Relación con el estudiante

Mapeo de la asociación — src/main/java/edu/proyecto/model/CuentaUsuario.java, líneas 11–12.

```
@OneToOne(optional=false, cascade=CascadeType.PERSIST) @JoinColumn(name="estudiante_id", nullable=false,
unique=true)
private Estudiante estudiante;
```

persist(cuenta) guarda también el estudiante por cascada PERSIST dentro de la misma transacción. Si el guardado falla, el código revierte la transacción; las ramas de error no se incluyen en la secuencia del flujo principal.

12 Incremento 2 CU02 Iniciar sesión

Precondición: cuenta registrada y credenciales correctas. El servicio compara la clave derivada mediante MessageDigest.isEqual. El Servlet establece cuentaId, estudianteld y nombreVisible en una sesión HTTP nueva.

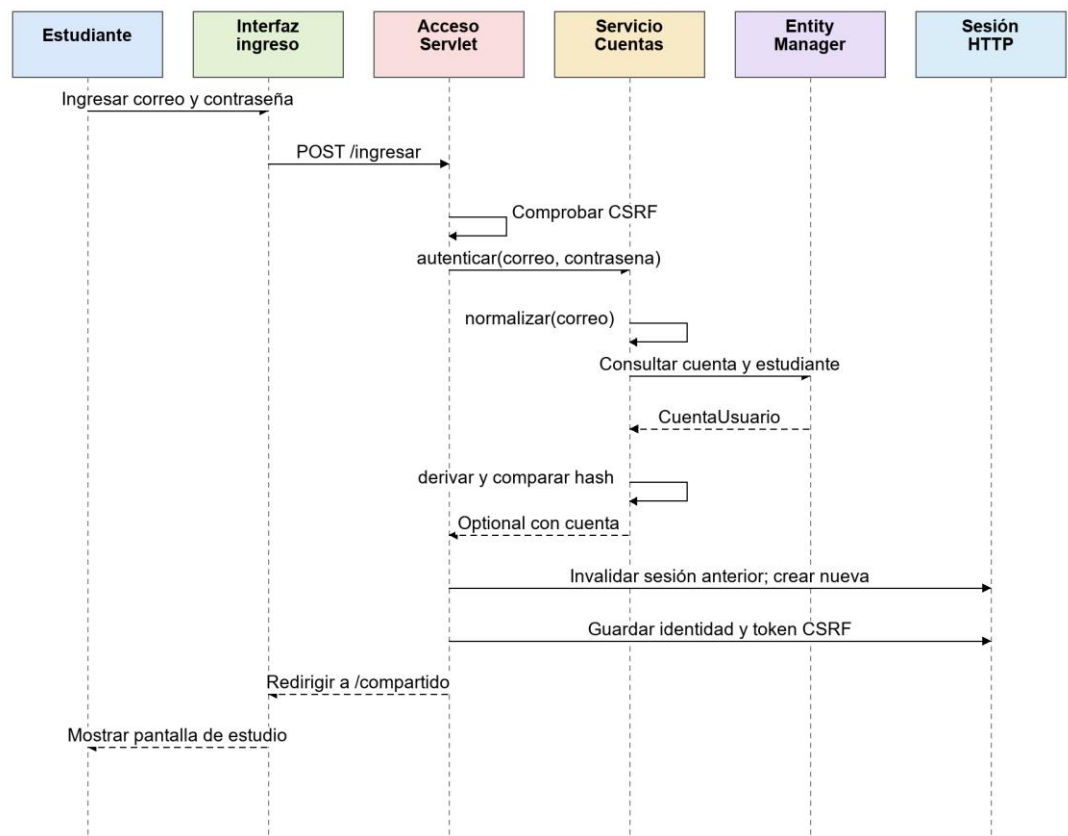


Figura 7. Secuencia del flujo principal. Fuente: modelo y código del proyecto.

Mensaje u operación	Ubicación en el código
autenticar(String, String)	ServicioCuentas.java:29
Sesión HTTP autenticada	AccesoServlet.java:46
Identidad y destino /compartido	AccesoServlet.java:47

AccesoFilter protege las pantallas de estudio. Una cookie con un UUID no sustituye la autenticación; IdentidadWeb obtiene al estudiante de la sesión del servidor.

13 Incremento 2 CU03 Emparejar compañero

Precondición: dos estudiantes autenticados disponibles; A ya solicitó emparejamiento y B inicia la solicitud que encuentra pareja. El servicio crea dos SesionEstudio. La sesión compartida construye dos participaciones e inicia ambos bloques con el mismo instante.

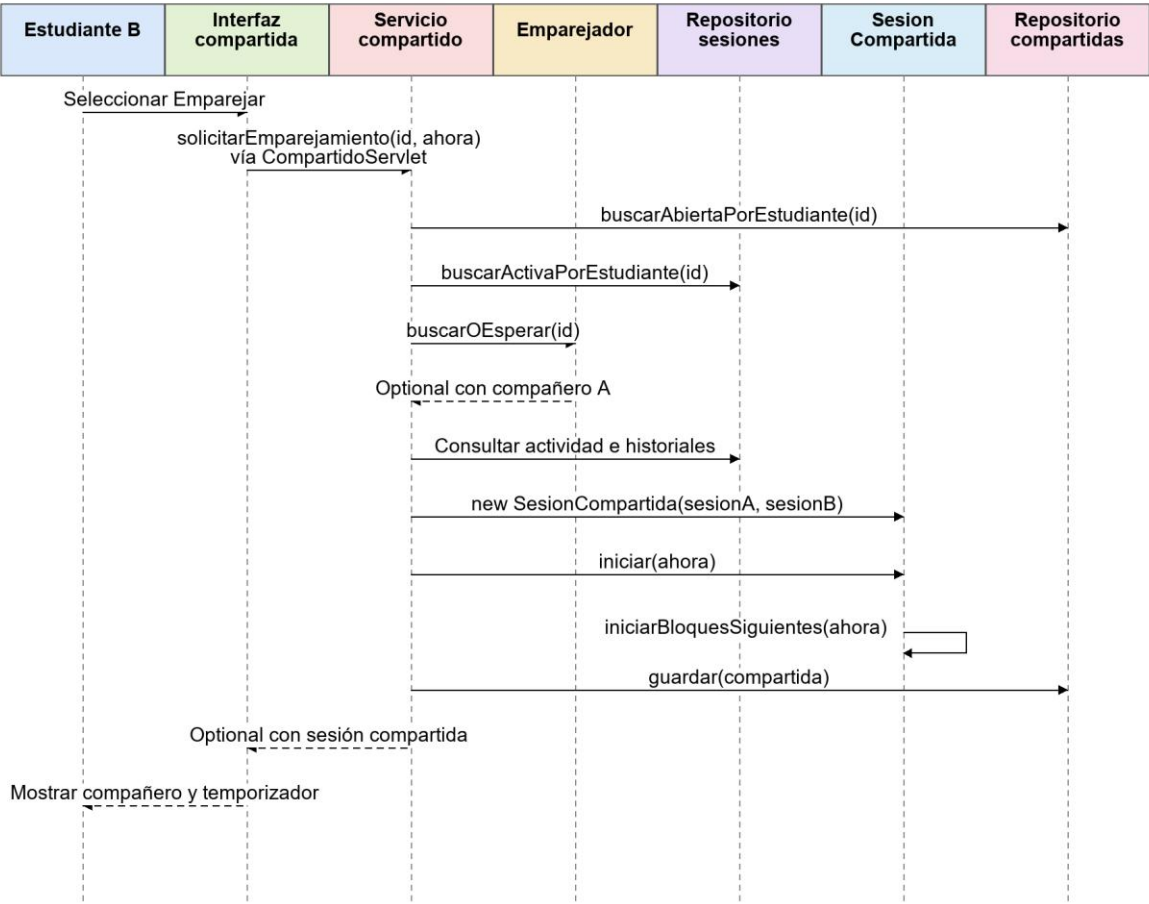


Figura 8. Secuencia del flujo principal. Fuente: modelo y código del proyecto.

Mensaje u operación	Ubicación en el código
solicitarEmparejamiento(UUID, Instant)	ServicioEstudioCompartido.java:20
buscarOEsperar(UUID)	EmparejadorEnMemoria.java:9
iniciar(Instant) e iniciarBloquesSiguietes	SesionCompartida.java:33

La interfaz de A detecta el emparejamiento mediante sus consultas periódicas. El flujo de espera previo se muestra en las evidencias; no se dibujan alternativas en esta secuencia.

14 Código y continuidad del estudio compartido

Creación e inicio de la composición — `src/main/java/edu/proyecto/service/ServicioEstudioCompartido.java`, líneas 38–41.

```
var compartida = new SesionCompartida(new SesionEstudio(a, configuracionFija, ahora), new SesionEstudio(b,
configuracionFija, ahora));
compartida.iniciar(ahora);
repositorioCompartidas.guardar(compartida); // Una única transacción para la sesión y ambas composiciones.
return Optional.of(compartida);
```

Conservación de los nombres

En el primer estudio, el servicio puede construir un Estudiante con nombre genérico si no hay historial. Antes de guardar, `RepositorioSesionesCompartidasJpa` consulta `CuentaUsuario` y recupera el nombre registrado. Esto evita sobrescribir la identidad del compañero con el nombre genérico.

Recuperación del nombre de cuenta — `src/main/java/edu/proyecto/repository/RepositorioSesionesCompartidasJpa.java`, líneas 15–16.

```
var cuenta = em.find(CuentaUsuario.class, individual.getEstudiante().getId());
if (cuenta != null)
individual.getEstudiante().cambiarNombreVisible(cuenta.getEstudiante().getNombreVisible());
```

Operaciones que mantienen la sesión

Operación del modelo	Efecto implementado
actualizarSesion → avanzarSiCorresponde	Completa bloques vencidos e inicia siguientes desde el instante real de vencimiento.
consultarTiempoRestante	Consulta el bloque de un participante presente; retorna <code>Optional<Duration></code> .
salir → registrarSalida(VOLUNTARIA)	Finaliza la sesión individual y registra la salida.
registrarDesconexion → registrarSalida(DESCONEXION)	Abandona la individual y registra la desconexión.
finalizar privado	Cierra la compartida cuando nadie sigue presente; ambas desconexiones producen <code>ABANDONADA</code> .

El detector de presencia utiliza 90 segundos sin señales y una revisión cada 10 segundos. Estas operaciones ya pertenecen al modelo; su explicación no agrega casos de uso al alcance de los siete documentados.

15 Evidencias del temporizador individual

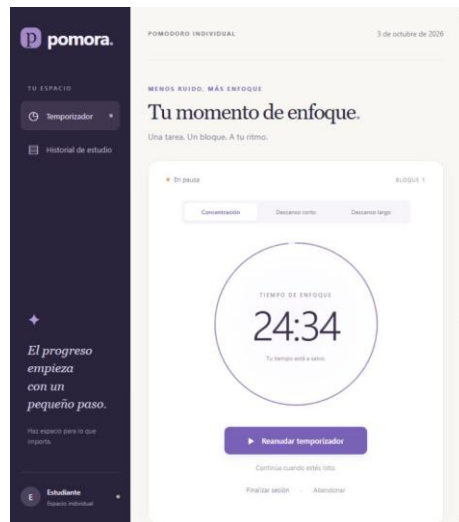


Figura 9. CU02: temporizador pausado. Captura histórica del repositorio; evidencia el estado y el tiempo restante.

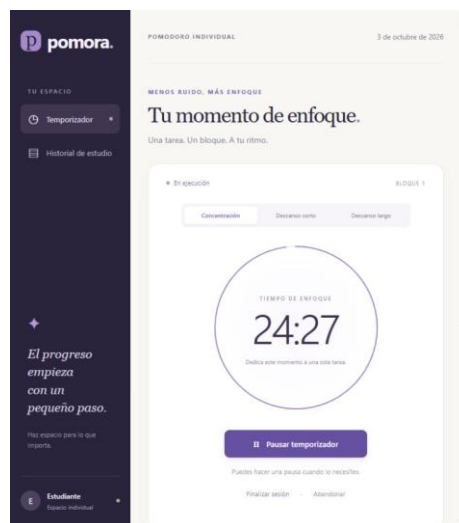


Figura 10. CU03: temporizador reanudado. Captura histórica de funcionamiento.

Los rótulos, colores y la navegación de estas capturas corresponden a la versión inicial. Las mejoras visuales posteriores no cambian los métodos de pausar y reanudar. Las pruebas actuales verifican la conservación del tiempo y la exclusión de los intervalos de pausa.

16 Evidencias del historial y las cuentas

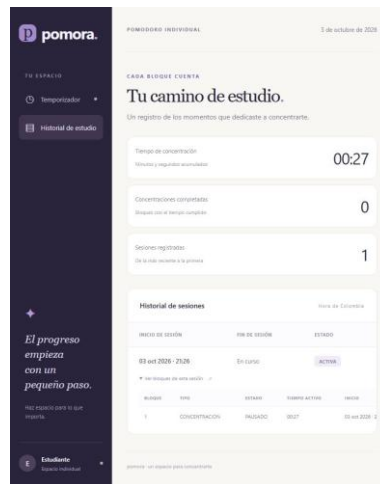


Figura 11. CU04 del incremento 1: historial con detalle de bloques. Captura histórica del repositorio.

pomora.

TU ESPACIO PARA CONCENTRARSE

Crea tu cuenta.

Tu nombre identificará tus secciones y será visible para tu compañero de estudio.

Nombre visible

Correo electrónico

Contraseña

Entre 8 y 128 caracteres.

Crear cuenta

>>

¿Ya tienes cuenta? Iniciar sesión

Figura 12. CU01 del incremento 2: formulario con nombre, correo y contraseña. Captura disponible del registro.

El nombre registrado se utiliza para identificar al estudiante y al compañero. La captura del formulario demuestra la entrada del caso de uso; el guardado, normalización y hash se verifican en CuentasTest y en los fragmentos de código anteriores.

17 Evidencias del ingreso y emparejamiento



The screenshot shows a mobile app interface for Pomora. At the top, it says "pomora." and "TU ESPACIO PARA CONCENTRARSE". Below that, it says "Bienvenido a Pomora." and "Inicia para estudiar, consultar tu historial y encontrar un compañero." There are two input fields: "Correo electrónico" and "Contraseña". Below these is a purple button labeled "Iniciar". At the bottom, there is a link that says "(¿Es tu primera vez? Regístrate)".

Figura 13. CU02 del incremento 2: formulario de ingreso con correo y contraseña.

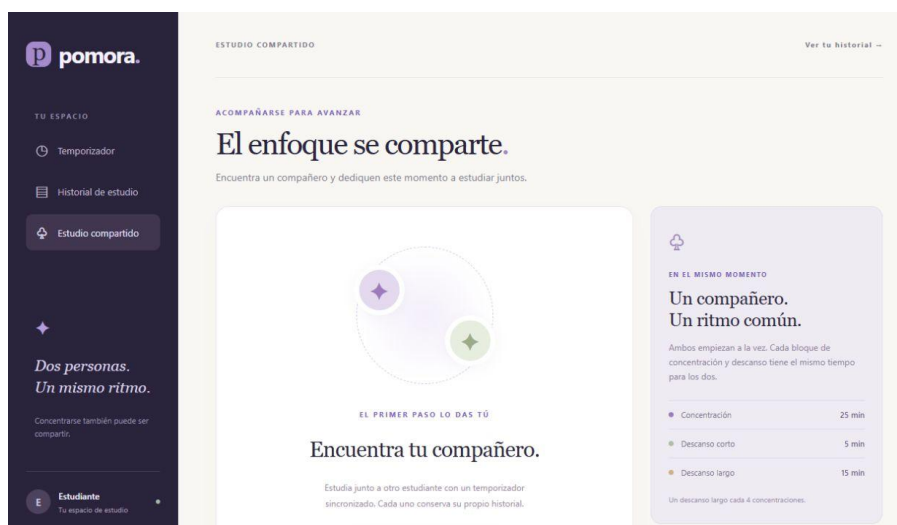


Figura 14. CU03 del incremento 2: pantalla de estudio compartido antes de encontrar pareja.

Las capturas se tomaron durante el desarrollo y se conservan en docs/evidencias. La pantalla compartida muestra la entrada al caso de uso; la persistencia de ambos nombres y el inicio sincronizado se comprueban mediante las pruebas de cuentas y estudio compartido. Estas imágenes no constituyen evidencias de IntelliJ IDEA ni de Copilot.

18 Verificación y resultados reproducibles

La revisión del 4 de octubre de 2026 ejecutó mvnw.cmd -o -B test. Maven finalizó con BUILD SUCCESS: 18 pruebas, cero fallos, cero errores y cero omitidas. Los reportes se conservan en target/surefire-reports; se usan bases H2 en memoria sin alterar las cuentas de la base local.

Clase de pruebas	Cantidad	Comportamiento cubierto
PomodoroDominioTest	6	Pausa, reanudación, vencimiento, ciclos, estados y validación.
PomodoroPersistenciaTest	4	Persistencia, composición, historial y exclusión de sesiones activas duplicadas.
EstudioCompartidoTest	6	Cola, sincronización, salidas, desconexión y participantes ajenos.
CuentasTest	2	Normalización, correo único, hash, autenticación y nombres persistidos.

Evidencia de los reportes

edu.proyecto.CuentasTest: Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 13.37 s -- in edu.proyecto.CuentasTest

edu.proyecto.EstudioCompartidoTest: Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.361 s -- in
edu.proyecto.EstudioCompartidoTest

edu.proyecto.PomodoroDominioTest: Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in
edu.proyecto.PomodoroDominioTest

edu.proyecto.PomodoroPersistenciaTest: Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.455 s -- in
edu.proyecto.PomodoroPersistenciaTest

Comprobaciones HTTP documentadas

docs/VERIFICACION.md registra 34 comprobaciones HTTP previas: 6 de acceso, 13 de compartido y 15 de Pomodoro. Ese resultado se distingue de las 18 pruebas Java ejecutadas en esta revisión; no se presenta como una nueva ejecución HTTP.

Reproducción

Con JDK 17 o superior: ejecutar preparar.ps1, iniciar.cmd y abrir http://localhost:8080/. Para generar el WAR: mvnw.cmd package. Con el servidor activo, ejecutar scripts/verificar-acceso.ps1, verificar-compartido.ps1 y verificar-web.ps1 con -BaseUrl. Estos scripts crean cuentas de prueba independientes.

19 Publicación y herramientas de desarrollo

Repositorio del proyecto

Nombre del proyecto Maven y del WAR: GR06_1BT3_622_26B. El número 06 corresponde al grupo indicado en la portada. Enlace informado por el grupo:

https://github.com/MayerliChavez/GR06_1BT3_622_26B

La entrega debe permitir acceder al código como repositorio público. La URL se incluye para revisión; esta comprobación local no certifica la visibilidad remota del repositorio.

Uso de IA y requisito de herramientas

El proyecto documenta asistencia de Codex en README.md y en el generador del informe anterior. La consigna exige GitHub Copilot e IntelliJ IDEA. La evidencia técnica de este informe acredita el código Java Web, los modelos, las pruebas y las pantallas; no acredita por sí misma el uso de esas dos herramientas.

La conformidad de ese requisito debe respaldarse con trabajo real y evidencias: abrir el proyecto Maven en IntelliJ IDEA, realizar y revisar una intervención con GitHub Copilot y conservar capturas que identifiquen el archivo, la propuesta y el resultado validado. No corresponde atribuir a Copilot el código asistido por otra herramienta.

Configuración para abrir el proyecto en IntelliJ IDEA

Abrir la carpeta del proyecto o pom.xml, importar la configuración Maven y seleccionar un JDK 17 o superior. Ejecutar la meta test para verificar el código y package para producir el WAR. La clase de entrada web depende del contenedor; iniciar.cmd utiliza Jetty mediante Maven. La tarea requiere además registrar la intervención real de Copilot en el IDE.

Documentación y modelo vigente

La referencia actual es el diagrama actualizado aportado por el grupo, que incorpora CuentaUsuario y ServicioCuentas. Las notas antiguas que describen esas clases como ausentes del diagrama original deben leerse como antecedentes de la versión anterior. Las secuencias de este informe corresponden al código revisado y al modelo actualizado.

Alcance de la verificación

Los diagramas muestran los flujos principales por decisión del grupo. Las pruebas incluyen condiciones inválidas para comprobar reglas, sin introducir escenarios alternativos en las figuras. Las comprobaciones realizadas no demuestran ausencia de todo posible defecto ni sustituyen la demostración en el entorno de entrega.

20 Conclusiones y bibliografía

Conclusiones

Los dos incrementos se materializan en una arquitectura Java Web con presentación JSP, control Servlet, servicios de aplicación y persistencia ORM. La implementación conserva las operaciones funcionales y relaciones del modelo actualizado, con adaptadores técnicos necesarios para HTTP y JPA.

La trazabilidad permite recorrer cada caso de uso desde su entrada hasta el método de dominio y su guardado. En el registro, normalizar y derivar pertenecen a ServicioCuentas; EntityManager realiza la consulta y la persistencia. En el estudio compartido, una composición reúne dos sesiones individuales y sus participaciones con un inicio común.

Las 18 pruebas Java aprobadas aportan evidencia de las reglas y de la persistencia. Las capturas muestran pantallas reales del desarrollo y se identifican las versiones históricas. La evidencia de IntelliJ IDEA, Copilot y visibilidad pública debe completarse para acreditar todos los requisitos de la consigna.

Bibliografía y fuentes técnicas

Grupo 06. Modelos de Pomora de la Tarea 1 y diagrama de clases actualizado suministrado para la Tarea 3.

Grupo 06. Código fuente de Pomora. `src/main/java`, `src/main/webapp` y `src/main/resources/META-INF/persistence.xml`.

Grupo 06. README.md; docs/INCREMENTO2.md; docs/TRAZABILIDAD.md; docs/VERIFICACION.md; docs/AUDITORIA-METODOS.md; docs/CAMBIOS-VISUALES.md.

Grupo 06. Reportes JUnit del 4 de octubre de 2026. `target/surefire-reports`. Capturas: `docs/evidencias`.

Object Management Group. Unified Modeling Language. <https://www.omg.org/spec/UML/>

Eclipse Foundation. Jakarta Servlet. <https://jakarta.ee/specifications/servlet/>

Eclipse Foundation. Jakarta Persistence. <https://jakarta.ee/specifications/persistence/>

Hibernate. Documentación Hibernate ORM. <https://hibernate.org/orm/documentation/>

GitHub. Documentación de GitHub Copilot. <https://docs.github.com/en/copilot>

JetBrains. IntelliJ IDEA. <https://www.jetbrains.com/idea/>